Chương 11

Bộ phân loại naive Bayes

11.1. Bộ phân loại naive Bayes

Xét các bài toán phân loại với C nhãn khác nhau. Thay vì tìm ra chính xác nhãn của mỗi điểm dữ liệu $\mathbf{x} \in \mathbb{R}^d$, ta có thể đi tìm xác suất để kết quả rơi vào mỗi nhãn: $p(y = c|\mathbf{x})$, hoặc viết gọn thành $p(c|\mathbf{x})$. Biểu thức này được hiểu là xác suất để đầu ra là nhãn c biết rằng đầu vào là vector $\mathbf{x}$. Nếu tính được biểu thức này, ta có thể giúp xác định nhãn của mỗi điểm dữ liệu bằng cách chọn ra nhãn có xác suất rơi vào cao nhất:

$$c = \underset{c \in \{1,\ldots,C\}}{\operatorname{argmax}}\, p(c|\mathbf{x}) \tag{11.1}$$

Nhìn chung, khó có cách tính trực tiếp $p(c|\mathbf{x})$. Thay vào đó, quy tắc Bayes thường được sử dụng:

$$c = \underset{c}{\operatorname{argmax}}\, p(c|\mathbf{x}) = \underset{c}{\operatorname{argmax}}\, \frac{p(\mathbf{x}|c)p(c)}{p(\mathbf{x})} = \underset{c}{\operatorname{argmax}}\, p(\mathbf{x}|c)p(c) \tag{11.2}$$

Dấu bằng thứ hai xảy ra theo quy tắc Bayes, dấu bằng thứ ba xảy ra vì $p(\mathbf{x})$ ở mẫu số không phụ thuộc vào c. Tiếp tục quan sát, $p(c)$ có thể được hiểu là xác suất để một điểm *bất kỳ* rơi vào nhãn c. Nếu tập huấn luyện lớn, $p(c)$ có thể được xác định bằng phương pháp ước lượng hợp lý cực đại (MLE) – là tỉ lệ giữa số điểm thuộc nhãn c và số điểm trong tập huấn luyện. Nếu tập huấn luyện nhỏ, giá trị này có thể được xác định bằng phương pháp ước lượng hậu nghiệm cực đại (MAP).

Thành phần còn lại $p(\mathbf{x}|c)$ là phân phối của các điểm dữ liệu trong nhãn c. Thành phần này thường rất khó tính toán vì $\mathbf{x}$ là một biến ngẫu nhiên nhiều chiều. Để có thể ước lượng được phân phối đó, tập huấn luyện phải rất lớn. Nhằm đơn giản

hoá việc tính toán, người ta thường giả sử rằng các thành phần của biến ngẫu nhiên $\mathbf{x}$ độc lập với nhau khi đã biết c:

$$p(\mathbf{x}|c) = p(x_1, x_2, \ldots, x_d|c) = \prod_{i=1}^{d} p(x_i|c) \tag{11.3}$$

Giả thiết các chiều của dữ liệu độc lập với nhau là quá chặt và trên thực tế, ít khi tìm được dữ liệu mà các thành phần hoàn toàn độc lập với nhau. Tuy nhiên, giả thiết *ngây thơ* (*naive*) này đôi khi mang lại những kết quả tốt bất ngờ. Giả thiết về sự độc lập của các chiều dữ liệu này được gọi là *naive Bayes*. Một phương pháp xác định nhãn của dữ liệu dựa trên giả thiết này có tên là *phân loại naive Bayes (NBC)*.

Nhờ giả thiết độc lập, NBC có tốc độ huấn luyện và kiểm tra rất nhanh. Việc này rất quan trọng trong các bài toán với dữ liệu lớn.

Ở bước huấn luyện, các phân phối $p(c)$ và $p(x_i|c), i = 1, \ldots, d$ được xác định dựa vào dữ liệu huấn luyện. Việc xác định các giá trị này có thể được thực hiện bằng MLE hoặc MAP.

Ở bước kiểm tra, nhãn của một điểm dữ liệu mới $\mathbf{x}$ được xác đinh bởi

$$c = \underset{c \in \{1,\ldots,C\}}{\operatorname{argmax}}\, p(c) \prod_{i=1}^{d} p(x_i|c). \tag{11.4}$$

Khi d lớn và các xác suất nhỏ, biểu thức ở vế phải của (11.4) là một số rất nhỏ, khi tính toán có thể gặp sai số. Để giải quyết việc này, (11.4) thường được viết lại dưới dạng tương đương bằng cách lấy log của vế phải:

$$c = \underset{c \in \{1,\ldots,C\}}{\operatorname{argmax}} \left(\log(p(c)) + \sum_{i=1}^{d} \log(p(x_i|c)) \right). \tag{11.5}$$

Việc này không ảnh hưởng tới kết quả vì log là một hàm đồng biến trên tập các số dương.

Sự đơn giản của NBC mang lại hiệu quả đặc biệt trong các bài toán phân loại văn bản, ví dụ bài toán lọc tin nhắn hoặc email rác. Trong phần sau của chương này, chúng ta cùng xây dựng một bộ lọc email rác tiếng Anh đơn giản. Cả quá trình huấn luyện và kiểm tra của NBC đều cực kỳ nhanh so với các phương pháp phân loại phức tạp khác. Việc giả sử các thành phần trong dữ liệu là độc lập với nhau khiến cho việc tính toán mỗi phân phối $p(\mathbf{x}_i|c)$ trở nên đơn giản.

Việc tính toán $p(\mathbf{x}_i|c)$ phụ thuộc vào loại dữ liệu. Có ba loại phân bố xác suất phổ biến là *Gaussian naive Bayes*, *multinomial naive Bayes*, và *Bernoulli Naive*. Chúng ta cùng xem xét từng loại.

11.2. Các phân phối thường dùng trong NBC

11.2.1. Gaussian naive Bayes

Mô hình này được sử dụng chủ yếu trong loại dữ liệu mà các thành phần là các biến liên tục. Với mỗi chiều dữ liệu i và một nhãn c, x_i tuân theo một phân phối chuẩn có kỳ vọng μ_{ci} và phương sai σ_{ci}^2:

$$p(x_i|c) = p(x_i|\mu_{ci}, \sigma_{ci}^2) = \frac{1}{\sqrt{2\pi\sigma_{ci}^2}} \exp\left(-\frac{(x_i - \mu_{ci})^2}{2\sigma_{ci}^2}\right) \tag{11.6}$$

Trong đó, bộ tham số $\theta = \{\mu_{ci}, \sigma_{ci}^2\}$ được xác định bằng MLE[30] dựa trên các điểm trong tập huấn luyện thuộc nhãn c.

11.2.2. Multinomial naive Bayes

Mô hình này chủ yếu được sử dụng trong bài toán phân loại văn bản mà vector đặc trưng được xây dựng dựa trên ý tưởng bag of words (BoW). Lúc này, mỗi văn bản được biểu diễn bởi một vector có độ dài d là số từ trong từ điển. Giá trị của thành phần thứ i trong mỗi vector là số lần từ thứ i xuất hiện trong văn bản đó. Khi đó, $p(x_i|c)$ tỉ lệ với tần suất từ thứ i (hay đặc trưng thứ i trong trường hợp tổng quát) xuất hiện trong các văn bản có nhãn c. Giá trị này có thể được tính bởi

$$\lambda_{ci} = p(x_i|c) = \frac{N_{ci}}{N_c}. \tag{11.7}$$

Trong đó:

- N_{ci} là tổng số lần từ thứ i xuất hiện trong các văn bản của nhãn c. Nó chính là tổng tất cả thành phần thứ i của các vector đặc trưng ứng với nhãn c.
- N_c là tổng số từ, kể cả lặp, xuất hiện trong nhãn c. Nói cách khác, N_c là tổng độ dài của tất cả các văn bản thuộc nhãn c. Có thể suy ra rằng $N_c = \sum_{i=1}^{d} N_{ci}$, từ đó $\sum_{i=1}^{d} \lambda_{ci} = 1$.

Cách tính này có một hạn chế là nếu có một từ mới chưa bao giờ xuất hiện trong nhãn c thì biểu thức (11.7) sẽ bằng không, dẫn đến vế phải của (11.4) bằng không bất kể các giá trị còn lại lớn thế nào (xem thêm ví dụ ở mục sau). Để giải quyết việc này, một kỹ thuật được gọi là *làm mềm Laplace* (Laplace smoothing) được áp dụng:

$$\hat{\lambda}_{ci} = \frac{N_{ci} + \alpha}{N_c + d\alpha} \tag{11.8}$$

với α là một số dương, thường bằng 1, để tránh trường hợp tử số bằng không. Mẫu số được cộng với $d\alpha$ để đảm bảo tổng xác suất $\sum_{i=1}^{d} \hat{\lambda}_{ci} = 1$. Như vậy, mỗi nhãn c được mô tả bởi một bộ các số dương có tổng bằng 1: $\hat{\lambda}_c = \{\hat{\lambda}_{c1}, \ldots, \hat{\lambda}_{cd}\}$.

[30] Xem ví dụ trang 71.

11.2.3. Bernoulli Naive Bayes

Mô hình này được áp dụng cho các loại dữ liệu mà mỗi thành phần là một giá trị nhị phân – bằng 0 hoặc 1. Ví dụ, cũng với loại văn bản nhưng thay vì đếm tổng số lần xuất hiện của một từ trong văn bản, ta chỉ cần quan tâm từ đó có xuất hiện hay không.

Khi đó, $p(x_i|c)$ được tính bởi

$$p(x_i|c) = p(i|c)^{x_i}(1 - p(i|c))^{1-x_i} \tag{11.9}$$

với $p(i|c)$ được hiểu là xác suất từ thứ i xuất hiện trong các văn bản của class c, x_i bằng 1 hoặc 0 tuỳ vào việc từ thứ i có xuất hiện hay không.

11.3. Ví dụ

11.3.1. Bắc hay Nam

Giả sử trong tập huấn luyện có các văn bản d1, d2, d3, d4 như trong Bảng 11.1. Mỗi văn bản này thuộc vào một trong hai nhãn: B (*Bắc*) hoặc N (*Nam*). Hãy xác định nhãn của văn bản d5.

Bảng 11.1: Ví dụ về nội dung của các văn bản trong bài toán Bắc hay Nam

	Văn bản	**Nội dung**	**Nhãn**
Dữ liệu huấn luyện	d1	hanoi pho chaolong hanoi	B
	d2	hanoi buncha pho omai	B
	d3	pho banhgio omai	B
	d4	saigon hutiu banhbo pho	N
Dữ liệu kiểm tra	d5	hanoi hanoi buncha hutiu	?

Ta có thể dự đoán rằng d5 có nhãn *Bắc*.

Bài toán này có thể được giải quyết bằng NBC sử dụng multinomial Naive Bayes hoặc Bernoulli naive Bayes. Chúng ta sẽ cùng làm ví dụ với mô hình thứ nhất và triển khai code cho cả hai mô hình. Việc mô hình nào tốt hơn phụ thuộc vào mỗi bài toán. Ta có thể thử cả hai để chọn ra mô hình tốt hơn.

Nhận thấy rằng ở đây có hai nhãn B và N, ta cần đi tìm $p(\text{B})$ và $p(\text{N})$ dựa trên tần số xuất hiện của mỗi nhãn trong tập huấn luyện. Ta có

$$p(\text{B}) = \frac{3}{4}, \qquad p(\text{N}) = \frac{1}{4}. \tag{11.10}$$

Tập hợp toàn bộ các từ trong tập huấn luyện là

$$V = \{\text{hanoi, pho, chaolong, buncha, omai, banhgio, saigon, hutiu, banhbo}\}$$

Pha huấn luyện

nhãn = B

	hanoi	pho	chaolong	buncha	omai	banhgio	saigon	hutiu	banhbo	
d1: $\mathbf{x}_1$	2	1	1	0	0	0	0	0	0	
d2: $\mathbf{x}_2$	1	1	0	1	1	0	0	0	0	
d3: $\mathbf{x}_3$	0	1	0	0	1	1	0	0	0	$d = \|V\| = 9$
Tổng	3	3	1	1	2	1	0	0	0	$\Rightarrow N_{\mathrm{B}} = 11$
$\Rightarrow \hat{\lambda}_{\mathrm{B}}$	4/20	4/20	2/20	2/20	3/20	2/20	1/20	1/20	1/20	$(20 = N_{\mathrm{B}} + \|V\|)$

nhãn = N

d4: $\mathbf{x}_4$	0	1	0	0	0	0	1	1	1	$\Rightarrow N_{\mathrm{N}} = 4$
$\Rightarrow \hat{\lambda}_{\mathrm{N}}$	1/13	2/13	1/13	1/13	1/13	1/13	2/13	2/13	2/13	$(13 = N_{\mathrm{N}} + \|V\|)$

Pha kiểm tra

d5: $\mathbf{x}_5 = [2, 0, 0, 1, 0, 0, 0, 1, 0]$

$$p(\mathrm{B}|\mathrm{d5}) \propto p(\mathrm{B}) \prod_{i=1}^{d} p(x_i|\mathrm{B})$$
$$= \frac{3}{4}\left(\frac{4}{20}\right)^2 \frac{2}{20}\frac{1}{20} \approx 1.5 \times 10^{-4}$$

$$p(\mathrm{N}|\mathrm{d5}) \propto p(\mathrm{N}) \prod_{i=1}^{d} p(x_i|\mathrm{N})$$
$$= \frac{1}{4}\left(\frac{1}{13}\right)^2 \frac{1}{13}\frac{2}{13} \approx 1.75 \times 10^{-5}$$

$\Rightarrow p(\mathbf{x}_5|\mathrm{B}) > p(\mathbf{x}_5|\mathrm{N}) \Rightarrow \mathrm{d5} \in \mathrm{nhãn(B)}$

Hình 11.1. Minh hoạ NBC với Multinomial naive Bayes cho bài toán *Bắc hay Nam*.

Tổng cộng số phần tử trong từ điển là $|V| = 9$.

Hình 11.1 minh hoạ quá trình huấn luyện và kiểm tra cho bài toán này khi sử dụng Multinomial naive Bayes, trong đó làm mềm Laplace được sử dụng với $\alpha = 1$. Chú ý, hai giá trị tìm được 1.5×10^{-4} và 1.75×10^{-5} không phải là hai xác suất cần tìm mà là hai đại lượng tỉ lệ thuận với hai xác suất đó. Để tính cụ thể, ta có thể làm như sau:

$$p(\mathrm{B}|\mathrm{d5}) = \frac{1.5 \times 10^{-4}}{1.5 \times 10^{-4} + 1.75 \times 10^{-5}} \approx 0.8955, \quad p(\mathrm{N}|\mathrm{d5}) = 1 - p(\mathrm{B}|\mathrm{d5}) \approx 0.1045.$$

Như vậy xác suất để d5 có nhãn B là 89.55%, có nhãn N là 10.45%. Bạn đọc có thể tự tính với ví dụ khác: d6 = pho hutiu banhbo. Nếu tính toán đúng, ta sẽ thu được

$$p(\mathrm{B}|\mathrm{d6}) \approx 0.29, \quad p(\mathrm{N}|\mathrm{d6}) \approx 0.71,$$

và suy ra d6 thuộc vào class N.

11.3.2. Bộ phân loại naive Bayes với thư viện scikit-learn

Để kiểm tra lại các phép tính phía trên, chúng ta cùng giải quyết bài toán này bằng scikit-learn. Ở đây, dữ liệu huấn luyện và kiểm tra đã được đưa về dạng vector đặc trưng sử dụng BoW.

```
from __future__ import print_function
from sklearn.naive_bayes import MultinomialNB
import numpy as np
# train data
d1 = [2, 1, 1, 0, 0, 0, 0, 0, 0]
d2 = [1, 1, 0, 1, 1, 0, 0, 0, 0]
d3 = [0, 1, 0, 0, 1, 1, 0, 0, 0]
d4 = [0, 1, 0, 0, 0, 0, 1, 1, 1]
train_data = np.array([d1, d2, d3, d4])
label = np.array(['B', 'B', 'B', 'N'])
# test data
d5 = np.array([[2, 0, 0, 1, 0, 0, 0, 1, 0]])
d6 = np.array([[0, 1, 0, 0, 0, 0, 0, 1, 1]])
## call MultinomialNB
model = MultinomialNB()
# training
model.fit(train_data, label)

# test
print('Predicting class of d5:', str(model.predict(d5)[0]))
print('Probability of d6 in each class:', model.predict_proba(d6))
```

Kết quả:

```
Predicting class of d5: B
Probability of d6 in each class: [[ 0.29175335  0.70824665]]
```

Kết quả này nhất quán với những kết quả được tính bằng tay ở trên.

Nếu sử dụng Bernoulli naive Bayes, chúng ta cần thay đổi một chút về feature vector. Lúc này, các giá trị khác không sẽ đều được đưa về một vì ta chỉ quan tâm đến việc từ đó có xuất hiện trong văn bản hay không.

```
from __future__ import print_function
from sklearn.naive_bayes import BernoulliNB
import numpy as np
# train data
d1 = [1, 1, 1, 0, 0, 0, 0, 0, 0]
d2 = [1, 1, 0, 1, 1, 0, 0, 0, 0]
d3 = [0, 1, 0, 0, 1, 1, 0, 0, 0]
d4 = [0, 1, 0, 0, 0, 0, 1, 1, 1]
train_data = np.array([d1, d2, d3, d4])
label = np.array(['B', 'B', 'B', 'N']) # 0 - B, 1 - N
# test data
d5 = np.array([[1, 0, 0, 1, 0, 0, 0, 1, 0]])
d6 = np.array([[0, 1, 0, 0, 0, 0, 0, 1, 1]])
## call MultinomialNB
model = BernoulliNB()
# training
model.fit(train_data, label)
# test
print('Predicting class of d5:', str(model.predict(d5)[0]))
print('Probability of d6 in each class:', model.predict_proba(d6))
```

Kết quả:

```
Predicting class of d5: B
Probability of d6 in each class: [[ 0.16948581  0.83051419]]
```

Ta thấy rằng, với bài toán nhỏ này, cả hai mô hình đều cho kết quả giống nhau (xác suất tìm được khác nhau nhưng không ảnh hưởng tới quyết định cuối cùng).

11.3.3. Bộ phân loại naive Bayes cho bài toán lọc email rác

Tiếp theo, chúng ta cùng làm việc với một bộ cơ sở dữ liệu lớn hơn[31]. Trong ví dụ này, dữ liệu đã được xử lý, và là một tập con của cơ sở dữ liệu *Ling-Spam dataset* (https://goo.gl/whHCd9).

Tập dữ liệu này bao gồm 960 email tiếng Anh, được tách thành tập huấn luyện và tập kiểm tra theo tỉ lệ 700:260 với 50% trong mỗi tập là các email rác.

Dữ liệu ở đây đã được tiền xử lý. Các quy tắc xử lý như sau[32]:

- *Loại bỏ stop words*: Những từ xuất hiện thường xuyên như 'and', 'the', 'of',... được loại bỏ vì chúng xuất hiện ở cả hai nhãn.
- *Lemmatization*: Những từ có cùng gốc được đưa về cùng loại. Ví dụ, 'include', 'includes', 'included' đều được đưa chung về 'include'. Tất cả các từ cũng đã được đưa về dạng ký tự thường.
- *Loại bỏ non-words*: chữ số, dấu câu và các ký tự đặc biệt đã được loại bỏ.

Dưới đây là một ví dụ của một email bình thường trước khi được xử lý:

```
Subject: Re: 5.1344 Native speaker intuitions

The discussion on native speaker intuitions has been extremely interesting,
but I worry that my brief intervention may have muddied the waters. I
take it that there are a number of separable issues. The first is the
extent to which a native speaker is likely to judge a lexical string as
grammatical or ungrammatical per se. The second is concerned with the
relationships between syntax and interpretation (although even here the
distinction may not be entirely clear cut).
```

[31] Dữ liệu trong ví dụ này được lấy từ *Exercise 6: Naive Bayes – Machine Learning, Andrew Ng* (https://goo.gl/kbzR3d).

[32] sử dụng thư viện *NLTK* (http://www.nltk.org/)

và sau khi được xử lý:

```
re native speaker intuition discussion native speaker intuition extremely
interest worry brief intervention muddy waters number separable issue first
extent native speaker likely judge lexical string grammatical ungrammatical
per se second concern relationship between syntax interpretation although
even here distinction entirely clear cut
```

Dưới đây là một ví dụ về email rác sau khi được xử lý:

```
financial freedom follow financial freedom work ethic extraordinary desire
earn least per month work home special skills experience required train
personal support need ensure success legitimate homebased income opportunity
put back control finance life ve try opportunity past fail live promise
```

Các từ 'financial', 'extraordinary', 'earn', 'opportunity',... là những từ thường thấy trong các email rác.

Trong ví dụ này, chúng ta sẽ sử dụng multinomial naive Bayes.

Để bài toán được đơn giản, chúng ta sẽ sử dụng dữ liệu đã được xử lý, có thể tải về tại https://goo.gl/CSMxHU. Thư mục sau khi giải nén bao gồm các file:

```
test-features.txt
test-labels.txt
train-features-50.txt
train-features-100.txt
train-features-400.txt
train-features.txt
train-labels-50.txt
train-labels-100.txt
train-labels-400.txt
train-labels.txt
```

tương ứng với các file chứa dữ liệu của tập huấn luyện và tập kiểm tra. File `train-features-50.txt` chứa dữ liệu của tập huấn luyện thu gọn với chỉ tổng cộng 50 email. Mỗi file `labels.txt` chứa nhiều dòng, mỗi dòng là một ký tự `0` hoặc `1` thể hiện email là thường hay rác.

Mỗi file `features.txt` chứa nhiều dòng, mỗi dòng có ba số, chẳng hạn:

```
1 564 1
1 19 2
```

Trong đó, số đầu tiên là chỉ số của email, bắt đầu từ 1; số thứ hai là thứ tự của từ trong từ điển (tổng cộng 2500 từ); số thứ ba là tần xuất của từ đó trong email đang xét. Dòng đầu tiên nói rằng trong email thứ nhất, từ thứ 564 trong từ điển

xuất hiện một lần. Cách lưu dữ liệu này giúp tiết kiệm bộ nhớ vì các email chỉ chứa một lượng nhỏ các từ trong từ điển.

Nếu biểu diễn mỗi email bằng một vector hàng có độ dài bằng độ dài từ điển (2500) thì dòng thứ nhất nói rằng đặc trưng thứ 564 của vector này bằng 1. Tương tự, đặc trưng thứ 19 của vector này bằng 2. Nếu không xuất hiện, các thành phần khác được hiểu bằng 0. Dựa trên các thông tin này, ta có thể tiến hành lập trình với thư viện sklearn.

Khai báo thư viện và đường dẫn tới files:

```
from __future__ import print_function
import numpy as np
from scipy.sparse import coo_matrix # for sparse matrix
from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.metrics import accuracy_score # for evaluating results
# data path and file name
path = 'ex6DataPrepared/'
train_data_fn = 'train-features.txt'
test_data_fn = 'test-features.txt'
train_label_fn = 'train-labels.txt'
test_label_fn = 'test-labels.txt'
```

Tiếp theo ta cần viết hàm số đọc dữ liệu từ file `data_fn` với nhãn tương ứng được lưu trong file `label_fn`. Chú ý rằng số lượng từ trong từ điển là 2500.

Dữ liệu được lưu trong một ma trận mà mỗi hàng là một vector đặc trưng của email. Đây là một ma trận thưa nên ta sử dụng hàm `scipy.sparse.coo_matrix`.

```
nwords = 2500

def read_data(data_fn, label_fn):
    ## read label_fn
    with open(path + label_fn) as f:
        content = f.readlines()
    label = [int(x.strip()) for x in content]
    ## read data_fn
    with open(path + data_fn) as f:
        content = f.readlines()
    # remove '\n' at the end of each line
    content = [x.strip() for x in content]
    dat = np.zeros((len(content), 3), dtype = int)
    for i, line in enumerate(content):
        a = line.split(' ')
        dat[i, :] = np.array([int(a[0]), int(a[1]), int(a[2])])

    # remember to -1 at coordinate since we're in Python
    data = coo_matrix((dat[:, 2], (dat[:, 0] - 1, dat[:, 1] - 1)),\
                      shape=(len(label), nwords))
    return (data, label)
```

Đoạn code dưới đây giúp lấy dữ liệu huấn luyện và kiểm tra, sau đó sử dụng `MultinomialNB` để phân loại.

```
(train_data, train_label)  = read_data(train_data_fn, train_label_fn)
(test_data, test_label)  = read_data(test_data_fn, test_label_fn)

clf = MultinomialNB()
clf.fit(train_data, train_label)

y_pred = clf.predict(test_data)
print('Training size = %d, accuracy = %.2f%%' % \
(train_data.shape[0],accuracy_score(test_label, y_pred)*100))
```

Kết quả:

```
Training size = 700, accuracy = 98.08%
```

Như vậy, có tới 98.08% email được phân loại đúng. Chúng ta tiếp tục thử với các tập huấn luyện nhỏ hơn:

```
train_data_fn = 'train-features-100.txt'
train_label_fn = 'train-labels-100.txt'
test_data_fn = 'test-features.txt'
test_label_fn = 'test-labels.txt'

(train_data, train_label)  = read_data(train_data_fn, train_label_fn)
(test_data, test_label)  = read_data(test_data_fn, test_label_fn)
clf = MultinomialNB()
clf.fit(train_data, train_label)
y_pred = clf.predict(test_data)
print('Training size = %d, accuracy = %.2f%%' % \
(train_data.shape[0],accuracy_score(test_label, y_pred)*100))

train_data_fn = 'train-features-50.txt'
train_label_fn = 'train-labels-50.txt'
test_data_fn = 'test-features.txt'
test_label_fn = 'test-labels.txt'

(train_data, train_label)  = read_data(train_data_fn, train_label_fn)
(test_data, test_label)  = read_data(test_data_fn, test_label_fn)
clf = MultinomialNB()
clf.fit(train_data, train_label)
y_pred = clf.predict(test_data)
print('Training size = %d, accuracy = %.2f%%' % \
(train_data.shape[0],accuracy_score(test_label, y_pred)*100))
```

Kết quả:

```
Training size = 100, accuracy = 97.69%
Training size = 50, accuracy = 97.31%
```

Ta thấy rằng thậm chí khi tập huấn luyện là rất nhỏ, chỉ có tổng cộng 50 email, kết quả đạt được đã rất ấn tượng.

Nếu bạn muốn tiếp tục thử mô hình `BernoulliNB`:

```
clf = BernoulliNB(binarize = .5)
clf.fit(train_data, train_label)
y_pred = clf.predict(test_data)
print('Training size = %d, accuracy = %.2f%%' % \
      (train_data.shape[0],accuracy_score(test_label, y_pred)*100))
```

Kết quả:

```
Training size = 50, accuracy = 69.62%
```

Ta thấy rằng trong bài toán này, `MultinomialNB` hoạt động hiệu quả hơn.

11.4. Thảo luận

11.4.1. Tóm tắt

- Bộ phân loại Naive Bayes (NBC) thường được sử dụng trong các bài toán phân loại văn bản.
- NBC có thời gian huấn luyện và kiểm tra rất nhanh. Điều này có được là do giả sử về tính độc lập giữa các thành phần.
- Nếu giả sử về tính độc lập được thoả mãn (dựa vào bản chất của dữ liệu), NBC được cho là sẽ có kết quả tốt hơn so với SVM (Phần VIII) và hồi quy logistic (Chương 14) khi có ít dữ liệu huấn luyện.
- NBC có thể hoạt động với các vector đặc trưng mà một phần là liên tục (sử dụng Gaussian naive Bayes), phần còn lại ở dạng rời rạc (sử dụng multinomial hoặc Bernoulli). Sự độc lập giữa các đặc trưng khiến NBC có khả năng này.
- Làm mềm Laplace được sử dụng để tránh trường hợp một từ trong tập kiểm tra chưa xuất hiện trong tập huấn luyện.
- Mã nguồn trong chương này có thể được tìm thấy tại https://goo.gl/yUR55o.

11.4.2. Đọc thêm

a. *Text Classification and Naive Bayes - Stanford* (https://goo.gl/HcefLX).

b. *6 Easy Steps to Learn Naive Bayes Algorithm (with code in Python)* (https://goo.gl/odQaaY).